

Git Happens

githappens4swe@gmail.com

Definizione del Way of Working Data: 28/10/2026

Informazioni documento

Redattori	F. Lissandrin
Verificatori	git Happens Team
Destinatari	git Happens Team
	Prof. T. Vardanega
	Prof. R. Cardin

Registro delle modifiche

Versione	Data	Autore	Verificatore	Dettaglio
0.0.1	2026/10/28	F. Lissandrin	G. Gambarro, C. Antoniazzo	prima stesura documento (sez. "Issue Tracking", "Procedura di contribuzione", e "Pubblicazione di documentazione")

1 Issue Tracking

Si definisce **Atlassian Jira** come principale strumento di issue tracking del team. **Jira** diventa inoltre unica fonte di verità sullo stato del progetto e delle attività in corso, il cui contenuto viene riassunto ed esportato nei verbali e diari di bordo del team.

Come visibile nelle procedure successivamente definite, *Jira* è al centro del lavoro del team.

1.1 Progetti (*spaces*) in Jira

Per ogni scope oggetto dell'attenzione del team, si crea uno *space* apposito in Jira. Lo space apposto all'amministrazione del team e la documentazione conseguente è identificato dal nome (e chiave) "*SWE*". Per lo space *SWE*, si sceglie la modalità "Kanban" di facile gestione e adatta alla natura dello space.

A partire dall'aggiudicazione del capitolato, viene creato uno spazio apposito al progetto in Jira. Lo spazio sarà utilizzato per la definizione di qualsiasi attività interna al progetto.

2 Contribuzione a Repositories

2.1 Procedura di contribuzione

Per contribuire ad un qualsiasi repository git HAPPENS, va seguita la procedura sottostante (specifica *"git flow"*):

1. Selezionare il branch base da cui effettuare le modifiche (solitamente *main* o il branch default)
2. Creare un branch a partire dal precedente. Nominarlo **KEY_ISSUE-NUMBER_ISSUE-DESC** dove *KEY* è la chiave del progetto Jira di riferimento, *ISSUE_NUMBER* è il numero della issue nel progetto, *ISSUE_DESC* è il nome dell'issue separato da trattini alti *"-"* e tutto in minuscolo (e.g. "SWE-37-bootstrap-del-way-of-working"). Vedere 2.3 per automatizzare la procedura.
3. Transizionare il ticket *Jira* in "In corso".
4. Lavorare al progetto suddividendo la contribuzione in opportune commit, vedi sez. 2.2 per la convenzione a riguardo.
5. Revisionare quanto svolto in autonomia, e successivamente **aprire una pull request**. Richiedere la revisione dei revisori assegnati e attendere.
6. Transizionare il ticket *Jira* in "In revisione".
7. I revisori controllano la PR (Pull Request), eventualmente approvandola o richiedendo modifiche.
8. Dopo aver ottenuto ogni approvazione e controllato il successo di eventuali automazioni, effettuare il *merge* della *PR*.
9. Transizionare il ticket *Jira* in "Completato".

2.2 Messaggi di commit

Ogni messaggio di commit sarà scritto in lingua **inglese** e seguendo la convenzione qui riportata: "semantic commit messages".

Scope Gli scope di commit sono un'indicazione di ciò che si è fatto in una operazione di commit ad alto livello, ad esempio:

- **feat:** lavoro su una feature di progetto.
- **docs:** aggiornamento / creazione di documentazione.
- **chore:** installazione pacchetti, organizzazione cartelle, pulizia repo, ...
- **refactor:** refactoring di codice **senza** modifiche di funzionalità.
- **test:** scrittura,revisione,... di test unitari/di integrazione/...

È buona prassi dividere le commit che coinvolgono più scope in una commit per ogni scope piuttosto che inventare uno scope o richiedere che venga aggiunto.

2.3 Strumenti di supporto

2.3.1 Estensione Atlassian di VS Code

Tramite l'estensione ufficiale di Atlassian per VS Code, è possibile utilizzare una comoda interfaccia grafica per eseguire i passi 2 e 3 della procedura in modo automatico. Infatti, dopo aver effettuato l'accesso a Jira tramite l'estensione, sarà visibile una lista di task (vedi figura 1).

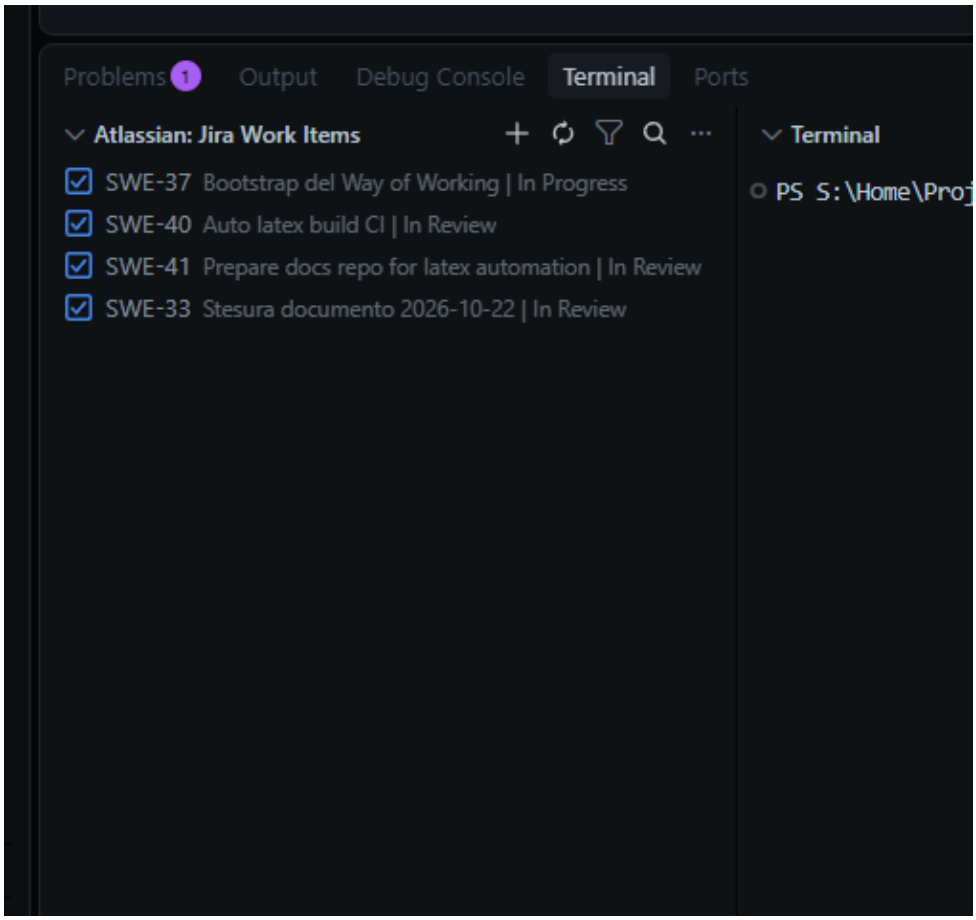


Figura 1: Lista di task nell'estensione

Selezionando un task, è possibile creare un branch (a partire da quello selezionato attualmente in locale) e cambiare lo stato del ticket in modo semplice e dallo stesso menù. La convenzione

di naming del branch rispecchia in automatico quella menzionata nella procedura. L'interfaccia visibile dopo aver selezionato un task consente inoltre di vederne i dettagli impostati su Jira.

Per aprire un branch e transizionare la issue, utilizzare il pulsante *Start Work* e seguire la procedura di apertura branch proposta.

3 Pubblicazione di documentazione (sul repo di documentazione e nel sito web)

Riferirsi alla procedura alla sezione 2.1 per quanto riguarda l'utilizzo di git.

3.1 Creazione del documento

Per pubblicare qualsiasi documento relativo al team git HAPPENS, per prima cosa è necessario trovare (o creare) la cartella di destinazione nel repo *documents* (e.g. *verbali/esterni/yyyy-mm-dd* o *verbali/interni/yyyy-mm-dd*).

Intestazione e template Creare nella cartella un file *main.tex* e inserire la direttiva `\documentclass{gh}`. Questa direttiva importerà in automatico l'intestazione e il template di git HAPPENS.

Combinare in modo opportuno questi comandi per riempire i campi comuni del file (omettere i comandi non necessari):

Comando	Descrizione
`\tipodoc{...}`	Tipo o titolo del documento, riportato anche nel piè di pagina
`\datadoc{...}`	Data del documento
`\modalita{...}`	Modalità di svolgimento (es. Online - Google Meet)
`\orario{...}`	Orario di inizio e fine
`\redattori{...}`	Redattori del documento
`\verificatori{...}`	Verificatori del documento
`\amministratore{...}`	Amministratore del gruppo
`\responsabile{...}`	Responsabile, riportato anche nel piè della prima pagina
`\destinatari{...}`	Destinatari del documento

Per inserire più valori in ogni campo, utilizzare il separatore "\\" (e.g. "F. Lissandrin \\ G. Gambarro").

Corpo del documento Inserire le direttive di corpo:

```
\begin{document}

\makeheader


\end{document}
```

Per ogni capitolo del file, creare un file in *capitoli/xx_ nome_ capitolo.tex* (dove *xx* è il numero progressivo di capitolo). Richiamare ogni capitolo inserendo l'istruzione `\input{capitoli/xx_nome_capitolo.` dopo l'istruzione *makeheader*.

3.1.1 Altre convenzioni file

- Se il documento deve contenere immagini, porle in una cartella **"assets"** allo stesso livello del file *main.tex*
- (TBA)

3.2 Pubblicazione del documento

1. (Opzionale) Creare un file *metadata.json* (vedi 3.2.1)
2. Facendo riferimento alla procedura 2.1, inviare il documento per la revisione.

Un workflow di GitHub Actions si occuperà di controllare la build, e al merge della *PR* un altro workflow effettuerà la pubblicazione sul sito web.

3.2.1 File metadata.json

Il file *metadata.json* definisce alcuni campi per migliorare la visualizzazione del file nel sito web.
Un file metadata è composto come segue:

```
{
    "title": "Titolo documento (e.g. 'Verbale esterno del 20 ottobre 2026')",
    "description": "Descrizione documento (e.g. 'Primo incontro con Sanmarco Informatica:
}
```

L'assenza di un file metadata non causa errori ma peggiora la visualizzazione del file nel sito (titolo dedotto dal nome file e descr. assente).

3.3 Strumenti di supporto

1. Estensione Latex Workshop per VS Code: consente di avviare una preview live dei documenti su cui si lavora. Prevede dei requisiti ulteriori da installare, riferirsi alla pagina dell'estensione.